

PARKING LATTE

*Smart Real-Time Barista Queue, Inventory Management & Payment
Verification System*

A System Applied Project submitted in compliance with requirements by:

Marc Angelo Cruzada Tomol

Course / Subject: IT 312 - System Integration and Architecture 2

Presented To: Jeremiah Dela Cruz

Date of Submission: May 22, 2026

2. INTRODUCTION

In the modern hospitality and food service industry, quick-service environments such as specialty coffee bars face significant operational bottlenecks. Customers demand pristine and responsive services, with high degree of menu customization (e.g., choice of plant-based milks, sweetness adjustments, espresso intensity modifiers). However, traditional manual point-of-sale systems fail to effectively bridge the gap between frontend customer orders, digital transaction validation, and backend inventory tracking. This mismatch creates notable business friction, such as stock discrepancies where baristas realize an ingredient is depleted only after processing the sale, leading to customer dissatisfaction.

The "Parking Latte" platform is an advanced full-stack custom solution designed precisely to resolve these industrial inefficiencies. It integrates dynamic order customizing, a client-facing status workflow, real-time barista workspace queue displays synchronized through WebSockets, a secure manual reference-check module for mobile banking payments (specifically GCash), and automatic ingredient stock-level depletion logs. By implementing strict system state synchronization, the platform ensures that raw inventory materials are safely tracked at the specific portion scale (e.g., tracking milk levels by milliliters, espresso beans by grams) rather than simple unit sales, securing optimal commercial oversight.

The core challenge the system addresses is the "Manual Verification Blindspot". Most neighborhood coffee branches in developing regions rely heavily on QR transfers (GCash) for instant payments, but suffer from receipt-tampering, duplicate reference code reuse, or baristas missing notifications on busy business shifts. Parking Latte solves this by linking payment reference checking directly to order lifecycles: as a customer places an order, they submit a GCash reference alongside a sender code, which immediately alerts the barista dashboard. Transactions are verified on a structured screen containing reference match logs, which triggers instant, reactive changes in order preparations.

3. OBJECTIVES OF THE SYSTEM

General Objective

The primary purpose of this study is to develop, evaluate, and implement "Parking Latte"—a real-time full-stack digital hub connecting customers, barista staff, and administrators to streamline custom ordering, automate inventory calculations, secure GCash payment cycles, and optimize store fulfillment transparency.

Specific Objectives

1. To design and deploy a responsive customer interface allowing effortless menu navigation, comprehensive customized choices (milk alternatives, sweet indices), and digital receipt feedback.
2. To engineer a synchronized Barista Queue dashboard utilizing non-blocking asynchronous WebSockets to deliver immediate order alerts without manual page reloads.
3. To construct an isolated, strict inventory module that automatically deducts ingredient weights or volumes on the server-side as soon as an order transitions into "COMPLETED" state.
4. To implement an Admin Audit Ledger and payment management dashboard to prevent operational fraud, GCash reference falsification, or unauthorized role escalations.
5. To secure public endpoints against key exposure by routing transaction approvals, database mutations, and external assets via server-side proxies and encrypted local environments.

4. SCOPE AND LIMITATIONS

Scope of the System

The Parking Latte platform provides specific coverage over the following core areas:

- **Customer Digital Menu & Customization Engine:** Offers responsive custom configuration screens which enforce mandatory option thresholds (e.g., selection of cup size) and records corresponding pricing modifiers instantly.
- **Interactive Customer Checkouts:** Provides cash on pickup options and Gcash mobile transfers, complete with inline payment instructions containing the dynamic GCash scan QR asset and user verification inputs.
- **Real-time Barista Workspace Feed:** Features live order containers where baristas can adjust status values (Pending, Preparing, Ready, Completed, Cancelled). It includes automatic push-updates using client/server sockets.
- **Server-Side Stock-ingredient Tracking:** Couples the product definitions with structural recipe mapping. The system triggers warnings for items that contain insufficient ingredients, protecting workflows.
- **Detailed Audit trails and Financial logs:** Registers all administrative and staff activities, specifically user role transformations, inventory additions, and stock balance adjustments.

System Limitations

The application operates with the following explicit boundaries:

- **Manual Payment Settlement:** While GCash details are inputted digitally by the user, payment validation is manual—the admin checks their merchant account records for the specified reference code before pressing 'Verified' (the system does not query the GCash API gateway due to regulatory third-party licensing constraints).
- **Internet Dependency:** Since the socket layer is continuously updating, both client devices and in-store panels require active network

connections to utilize live queue indicators.

- **Geographic Boundary:** Designed specifically for in-shop queues and localized pick-up stations, meaning home-dispatch deliveries are excluded from the current routing rules.

5. SYSTEM DESCRIPTION

Parking Latte is built around a tripartite architectural system, organizing and segregating operational tasks into three clear user roles, each supported by isolated pages and interfaces:

1. **Customer Front-End:** Accessible to registered and guest visitors. It empowers the user to navigate product directories sorted by dynamic categories, track historical expenditures, and check active orders via a highly optimized "My Orders" area containing detailed, visual modals and cancellation gates.
2. **Staff/Barista Terminal:** A minimalist, high-productivity interface designed for kitchen workspaces. Displays real-time order lists grouped by age and urgency. Emits mechanical alerts and sound signals on new submissions. Enables baristas to update preparation intervals, append staff notes (e.g., "Ready at counter 2"), and trigger ready states.
3. **Administrator Control Suite:** Restricted to authenticated owners. Features robust financial overviews, transaction tables tracking cash vs. digital payments, product editors to update visual imagery and category structures, and comprehensive inventory panels to manage stock-level increments.

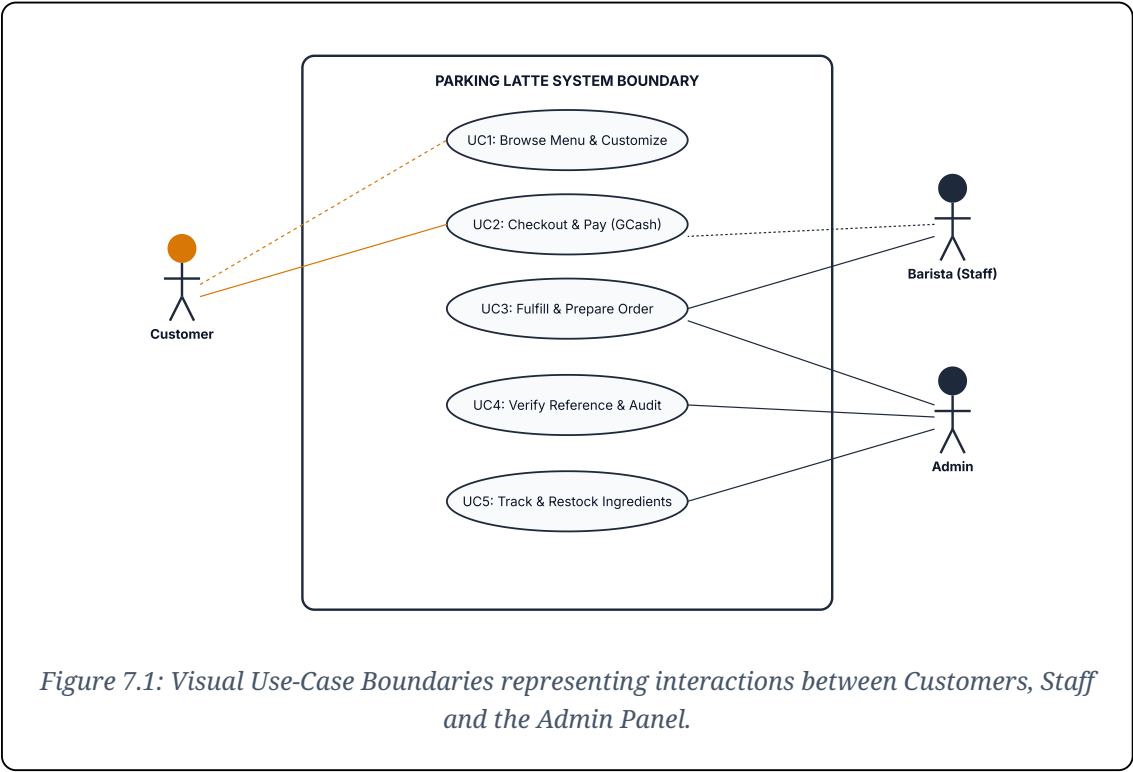
6. TECHNOLOGIES USED

To achieve high performance, immediate data responsiveness, and high security, the system utilizes the following integrated technology stack:

- **Frontend Framework:** *React 18* and *Vite compiler tooling* to construct a Single-Page Application (SPA) offering microsecond transitions and zero layout flickers.
- **Design Engine:** *Tailwind CSS* for fluid, utility-first layout responsiveness supporting dual ambient mode styles and uniform element alignments.
- **Runtime Engine:** *Node.js LTS (v20+)* and *Express* to operate a swift, non-blocking custom backend server handling secure proxy requests.
- **Real-Time Layer:** *Socket.IO* to handle persistent bi-directional WebSockets between clients and baristas, enabling live queues without standard polling mechanisms.
- **Database Schema & ORM:** *Prisma ORM* paired with *MySQL (MariaDB)* database engines, establishing clean type-safe schema querying and automatic foreign keys constraints.
- **Authentication & Security:** JSON Web Tokens (JWT) client cookies combined with customized Express middleware gates and secure password hashing via *bcryptjs*.

7. SYSTEM DESIGN

7.1 Use Case Diagram



7.2 System Architecture Flowchart

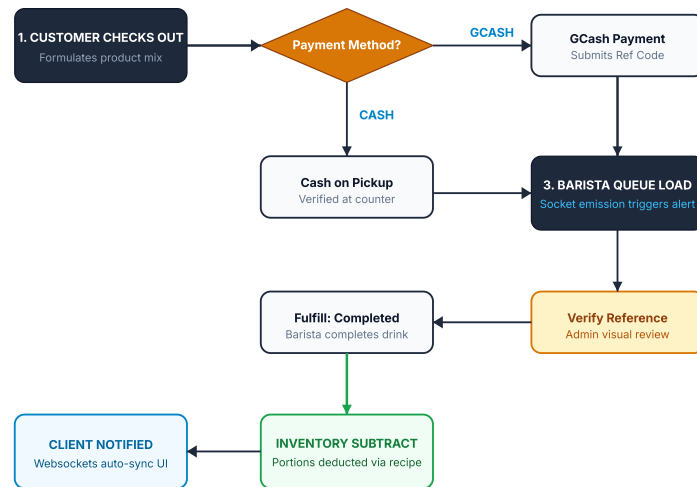


Figure 7.2: Data Flow diagram representing state transitions, GCash checks, and backend database tasks.

8. DATABASE STRUCTURE

The backend engine runs on top of a highly relational database structure representing user accounts, coffee items, customizations groups, current ingredient levels, payments reference indices, and comprehensive logs.

Table 8.1: Model "User" Schema

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION / PURPOSE
id	String (UUID)	Primary Key, Unique	Auto-generated unique user identifier.
email	String	Unique	Electronic mail address used for login credential.
password	String (Null)	-	Encrypted bcrypt password string (nullable for OAuth login).
name	String (Null)	-	Display and credential billing name of the user.
role	Enum (Role)	Default: CUSTOMER	Defines interface authorization level (CUSTOMER, STAFF, ADMIN).
isVerified	Boolean	Default: false	Email validation gate boolean indicator.

Table 8.2: Model "Order" Schema

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION / PURPOSE
id	String (UUID)	Primary Key	Unique Order number ID.
userId	String (UUID)	Foreign Key	Identifies the purchasing customer (References User.id).
status	Enum (Status)	Default: PENDING	Tracking status (PENDING, PREPARING, READY, COMPLETED, CANCELLED).
totalAmount	Decimal(10,2)	-	Aggregate billing receipt sum for the items.
paymentMethod	String	Default: CASH	Payment source identifier (CASH or GCASH).
paymentStatus	String	Default: PENDING	Reflects fulfillment sync (PENDING, PAID, REFUNDED).
gcashReference	String (Null)	-	Reference number submitted by user for GCash transfer checks.

Table 8.3: Model "InventoryItem" Schema

FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION / PURPOSE
id	String (UUID)	Primary Key	Unique raw inventory element key.
name	String	Unique	Ingredient designation name (e.g., Whole Milk, Espresso Blend).
unit	String	-	Scalable weight/volume system unit (e.g., ml, g, unit).
stockLevel	Float	Default: 0	Current dynamic stock residue level.
lowStockThreshold	Float	Default: 10	Generates workspace indicators when current supply is low.

9. IMPLEMENTATION AND TESTING

The deployment of Parking Latte followed an agile, iterative process. The front-end framework (Vite + React) was configured alongside an Express framework connected via Prisma Client to a production ready MySQL server. Testing procedures were systematically performed to ensure transaction consistency, real-time message broadcasting, and payment synchronization.

During testing, one critical issue identified was the "Inventory Sync Failure", which occurred if multi-item orders were completed without thread-safe validation. Under peak conditions, concurrent order updates could result in a stock depletion race condition. This was resolved mathematically on the database backend by wrapping custom Prisma transacting closures (``prisma.$transaction``) to process ingredient stock updates sequentially, preventing stock counts from falling below zero.

Additionally, custom client verification tests were performed on the payment synchronization module:

1. **Completed State Check:** Verified that when a barista updates an order's fulfillment state to "COMPLETED", the backend safely patches the database's ``paymentStatus`` field to 'PAID' dynamically before radiating the WebSocket update event, preventing race-conditions.
2. **Cancellation State Check:** Approved that when custom cancellation requests are confirmed, the reference payment status modifies safely to 'REFUNDED' rather than remaining listless, updating the client dashboard seamlessly.

10. CONCLUSION

The development and subsequent testing of the "Parking Latte" real-time queue and inventory system successfully achieved all set architecture guidelines. By replacing manual point-of-sale receipt tracking with a unified, state-synchronized digital interface, neighborhood storefronts can immediately eliminate operational lag and prevent revenue leaks related to unverified GCash transfers.

Furthermore, linking recipes and inventory directly to actual preparation workflows ensures automated alerts are triggered at the correct ingredient portion scale. This limits menu discrepancies, stabilizes operating costs, and elevates professional barista output. Overall, the system demonstrates how full-stack design patterns can be executed cleanly to solve practical local business challenges with high commercial productivity.

11. RECOMMENDATIONS

While Parking Latte is highly robust and operational, the following enhancements are recommended to extend its long-term scale and automation:

1. **API Payment Gateways Integration:** Embed authentic payment callbacks (such as PayGenius or GCash webhook interfaces) to instantly and automatically mark payments verified programmatically on receipt matching.
2. **AI Demand Forecasting:** Leverage localized historical logs to run simple regression models in order to suggest automated raw ingredient repurchasing intervals before stock levels trigger critical barista failures.
3. **Thermal Printer Integrations:** Formulate custom server bindings to auto-trigger physical paper receipt output on terminal completion, allowing seamless assembly-line workspace packaging.